接入文档

用 AK 和 SK 从你自己的程序调用开放接口。请求和响应都加密，能解密成功即表示认证通过，不需要另外签名或换取 token。

API 凭证

你的接入信息

Go 示例 | Python 示例 | 测试向量

接口地址
`https://op.privapi.cc`

Access Key
`gwak_9273db4f596910a8abc0f443`

频率限制
每个 AK 120 次/分钟
每个 IP 300 次/分钟

SK 在「API 凭证」页查看，只放在服务端程序里。下方示例已填入你的接口地址和 AK。

本页目录

密钥派生

快速开始

1. 在「API 凭证」开通凭证，勾选要调用的接口。
2. 下载 Go 或 Python 示例，运行 `selftest` 用测试向量核对密钥派生和加解密，不联网。自己实现时也先让测试向量对上，再联调。
3. 把示例里的 Client 复制进你的项目，SK 从环境变量读取，然后调用接口。

main.go Go Python 复制

```
// 把 gateway_open.go 里的 Client 复制到你的项目
const (
        L    = "https://op.privapi.cc"
Key = "gwak_9273db4f596910a8abc0f443"
)

client, err := NewClient(baseURL, accessKey, os.Getenv("GATEWAY_SK"))
```

跳转至主要内容

```
if err != nil {
    log.Fatal(err)
}
var balance struct {
    Balance string `json:"balance"`
}
err = client.Call(ctx, "/open/v1/balance/query", nil, &balance)
```

Go 示例需要 Go 1.24 及以上，只依赖标准库；Python 示例依赖 `cryptography`（`pip install cryptography`）。

凭证

在控制台「开放平台 → API 凭证」开通，得到一对 AK 和 SK。

名称	格式	说明
AK	`gwak_` + 24 位十六进制	公开标识，放在请求头里
SK	`gwsk_` + 43 位 base64url	密钥，只放在服务端程序里，不要放进浏览器、App 或代码仓库

- 每人只有一个凭证，同一时刻只有一个 SK 生效。SK 随时可以在凭证页查看。
- 重置 SK 后旧 SK 立即失效，AK 不变。多实例部署时请在低峰期重置，并尽快换上新 SK。
- 可以设置 IP 白名单和过期时间；停用、注销立即生效。
- 怀疑 SK 泄露时立即重置。

密钥派生

SK 去掉 `gwsk_` 前缀、按 base64url（无填充）解码得到 32 字节，用 HKDF-SHA256 派生两把 AES-256 key：

密钥派生 复制

```
req_key  = HKDF-SHA256(ikm = SK 的 32 字节, salt = AK 的 ASCII, info = "gw-open/v1/re
resp_key = HKDF-SHA256(ikm = SK 的 32 字节, salt = AK 的 ASCII, info = "gw-open/v1/re
```

跳转至主要内容

SK 不变则两把 key 不变，程序启动时算一次即可。

请求

请求示例 复制

```
POST /open/v1/balance/query
Content-Type: application/json
X-Gw-Access-Key: gwak_9273db4f596910a8abc0f443
X-Gw-Timestamp: 1792646400
X-Gw-Nonce: c4a1f09e2b7d4e6385a0d9f1b3e27c58

{"payload": "<base64(iv || ciphertext || tag)>"}
```

请求头	说明
X-Gw-Access-Key	AK
X-Gw-Timestamp	当前 Unix 时间（秒），与服务器相差不能超过 300 秒
X-Gw-Nonce	16–64 个 `[A-Za-z0-9_-]` 字符，**每个请求重新随机生成**，建议用 16 字节随机数的十六进制

- 所有接口都是 `POST`，完整地址为接口地址加路径；路径与文档完全一致，不带查询串和结尾斜杠。请求体不超过 32 KiB。
- 业务参数是 JSON 对象（没有参数时为 `{}`），用 `req_key` 加密后放进 `payload`。
- 加密算法为 AES-256-GCM：每条消息生成新的 12 字节随机 IV，tag 16 字节，`payload` = 标准 `base64(iv || ciphertext || tag)`。同一把 key 下 IV 绝不能重复。
- AAD 为以下各行用 `\n` 连接，末尾不加换行：

请求 AAD 复制

```
GW1-REQ
POST
{path}          例如 /open/v1/balance/query
{AK}
{X-Gw-Timestamp 的原始值}
{X-Gw-Nonce 的原始值}
```

方法、路径、AK、时间戳、nonce 和 body 任何一处被改动，服务端都会解密失败，返回

跳转至主要内容

响应

body 里有 `payload` 就用 `resp_key` 解密，没有就是明文错误。

认证阶段的错误：明文返回

明文错误 复制

```
{"code": "AUTH_FAILED", "message": "认证失败", "request_id": "…"}
```

通过认证后：成功和业务错误都加密返回

加密响应 复制

```
{"request_id": "…", "payload": "<base64(iv || ciphertext || tag)>"}
```

解密的 AAD 为 `GW1-RESP\n{AK}\n{本次请求的 nonce}`，解密后得到：

解密后的响应 复制

```
{"code": "OK", "message": "", "data": { … }}
```

AAD 绑定了请求的 nonce：只有本次请求的响应才能解开，可以据此确认响应没有被替换或重放。HTTP 状态码与 `code` 一一对应；报障时请提供 `request_id`。

错误码

code	HTTP	说明	是否加密
`INVALID_REQUEST`	400	缺少请求头、格式错误、信封不合法、body 超过 32 KiB、带查询串	否
`TIMESTAMP_EXPIRED`	401	时间戳与服务器相差超过 300 秒；响应头 X-Gw-Server-Time 为服务器时间	否
`:CESS_KEY`	401	AK 不存在、已注销、已停用、已过期，或账号已停用	否

跳转至主要内容

code	HTTP	说明	是否加密
AUTH_FAILED	401	解密失败：SK 不对（含已被重置的旧 SK）、派生或 AAD 拼接有误、请求被改动	否
NONCE_REPLAYED	401	nonce 已使用过	否
IP_NOT_ALLOWED	403	来源 IP 不在白名单中	否
RATE_LIMITED	429	超出频率限制，按 Retry-After 秒后重试	否
NOT_FOUND	404	接口不存在	否
SCOPE_DENIED	403	凭证没有勾选该接口，或账号已没有对应权限	是
INVALID_PARAMS	400	业务参数不合法，含未知字段	是
SESSION_NOT_FOUND	404	查询封禁原因时，该 session 没有经你的 Key 转发过请求	是
UNAVAILABLE	503	服务暂时不可用，稍后重试	视阶段而定

「是否加密」为否的错误在认证完成前返回，body 是明文。 `AUTH_FAILED` 无法区分具体原因，请先用测试向量核对实现。

频率限制

- 每个 AK 每分钟 120 次；每个来源 IP 每分钟 300 次，含失败的请求。
- 超出返回 `429 RATE_LIMITED` ，响应头 `Retry-After` 为建议的等待秒数。
- 数据按分钟级更新，轮询间隔建议不小于 30 秒。

接口

- 金额都是十进制字符串（美元，精度到 0.000001），请用十进制类型解析，不要用浮点数。
- 以后可能增加字段，请忽略不认识的字段。
- 凭证需要勾选对应接口，账号也要有对应权限，否则返回 `SCOPE_DENIED` 。

跳转至主要内容

查询余额 `/open/v1/balance/query` 已开启

查询节点	`/open/v1/nodes/query`	未勾选
查询请求日志	`/open/v1/logs/query`	未勾选
查询封禁原因	`/open/v1/bans/query`	未勾选

查询余额 已开启

`POST` `/open/v1/balance/query` scope `balance:read`

当前余额、停用阈值和今日、本周消费。

参数

查询余额的参数 复制

```
{}
```

返回的 data

查询余额的返回示例 复制

```
{
  "currency": "USD",
  "balance": "123.456789",
  "min_balance": "0",
  "suspended": false,
  "billing_exempt": false,
  "today_cost": "1.5",
  "week_cost": "12.25"
}
```

字段	说明
`balance`	当前余额
`min_balance`	停用阈值：余额降到该值或以下时 Key 会被停用
	是否因余额不足停用了 Key
`billing_exempt`	是否免计费

跳转至主要内容

字段	说明
today_cost / week_cost	今日、本周消费，按北京时间的自然日、自然周（周一开始）统计

查询节点 未勾选

POST /open/v1/nodes/query scope nodes:read

自己名下节点的状态、并发和上游额度。

参数

查询节点的参数 复制

```
{"node_ids": [], "page": 1, "page_size": 100}
```

- 参数都可以省略。
- node_ids 最多 100 个，只返回其中属于自己的节点；为空表示全部。
- page 从 1 开始； page_size 默认 100，最大 200。结果按 id 排序。

返回的 data

查询节点的返回示例 复制

```
{
  "total": 1,
  "items": [{
    "id": "a1b2c3d4e5f60718",
    "name": "pve-101",
    "remark": "",
    "platform": "linux",
    "enabled": true,
    "online": true,
    "state": "idle",
    "limits": {"max_concurrent": 5, "rpm_limit": 30},
    "sessions": {"count": 3, "total_slots": 10},
    "quota": {
      "today_requests": 1200,
          e_hour_utilization": 0.42, "five_hour_reset_at": "1792650000",
跳转至主要内容  en_day_utilization": 0.43, "seven_day_reset_at": "1792900000",
        ble_utilization": 0.91, "fable_reset_at": "1792900000", "fable_exhausted":
    },
    "updated_at": "2026-10-22T08:00:00Z"
```

```
  }]
}
```

字段	说明
`platform`	`linux` / `macos` / `windows` / `all`
`state`	调度状态；尚无采样时为 `unknown`
`limits`	并发上限与每分钟请求上限
`sessions`	当前会话数与总槽位
`quota.*_utilization`	上游额度占用率，0–1；未知为 `null`
`quota.*_reset_at`	额度重置时间（Unix 秒，字符串）；未知为 `null`
`quota.fable_exhausted`	Fable 周额度是否已用尽、等待重置

查询请求日志 未勾选

`POST` `/open/v1/logs/query` scope `logs:read`

按编号查自己 Key 的请求，每条都带详情：客户端请求头、错误说明和客户端实际收到的响应正文。

参数

查询请求日志的参数 复制

```
{
  "request_id": "0b6f5d7e-1c2a-4e3b-9f8d-7a6b5c4d3e2f",
  "upstream_request_id": "",
  "session_id": "",
  "from": 0,
  "to": 0,
  "cursor": "",
  "page_size": 20
}
```

跳转至主要内容

`d` 、 `upstream_request_id` 、 `session_id` 至少给一个，都是精确匹配；同时给出多个时，返回同时满足的请求。

- `from` / `to` ： 毫秒时间戳，含 `from` 、不含 `to` ； 0 或省略表示不限。

- `page_size` 默认 20，最大 50。结果按请求开始时间从新到旧排列。
- `cursor`：翻页时传上一页返回的 `next`，其他参数保持不变。

参数	来源
`request_id`	网关请求编号：模型接口响应头 `X-Request-ID`，也是错误响应体中的 `request_id`（UUID，大小写不限）
`upstream_request_id`	上游请求编号：模型接口响应头 `Request-Id`
`session_id`	客户端会话 ID，最长 256 字节

只能查到自己 Key 的请求；不是自己的请求与不存在的请求一样，返回空列表。

返回的 data

查询请求日志的返回示例 复制


```
{
  "next": "1792646400000.42",
  "items": [{
    "request_id": "0b6f5d7e-1c2a-4e3b-9f8d-7a6b5c4d3e2f",
    "upstream_request_id": "req_011CU…",
    "outcome": "error",
    "status": 429,
    "model": "claude-sonnet-4-5",
    "stream": true,
    "key": {"id": 3, "name": "prod-key"},
    "node": {"id": "a1b2c3d4e5f60718", "name": "pve-101"},
    "client": {
      "session_id": "4f1c…",
      "session_source": "x-claude-code-session-id",
      "agent_id": "",
      "client_request_id": "",
      "user_agent": "claude-cli/2.0.14 (external, cli)",
      "ip": "198.51.100.7",
      "headers": {"user-agent": "claude-cli/2.0.14 (external, cli)", "anthropic-ver
    },
    "error": {
      "type": "rate_limit_error",
      "code": "",
      "message": "…",
      "client_closed": false,
      "hint": {"title": "服务容量或频率受限", "message": "当前可用服务已达到容量或请求频率上
    },
    "response": {
      "body": "{\"type\":\"error\",…}",
      "encoding": "utf-8",
      "content_type": "application/json",
```


跳转至主要内容

```
      "content_encoding": "",
      "size": 152,
      "truncated": false,
      "complete": true
    },
    "usage": {"input_tokens": 0, "output_tokens": 0, "cache_read_tokens": 0, "cache
    "cost": null,
    "timing": {"ttfb_ms": 320, "ttft_ms": null, "duration_ms": 330},
    "request_bytes": 18234,
    "response_bytes": 152,
    "created_at": "2026-10-22T05:20:00Z",
    "finished_at": "2026-10-22T05:20:00.33Z"
  }]
}
```


字段	说明
next	下一页的 cursor ；为空表示已到最后一页
outcome	success 成功； error 已转发但失败； rejected 被拦截（Key、余额、额度、内容安全、会话封禁等），三者互不重叠
status	返回给客户端的 HTTP 状态码
node	处理请求的节点；未分配到节点时为 null
client	客户端随请求发来的标识； session_source 为会话 ID 的来源（请求头名或 metadata.user_id ）。 headers 只含客户端标识类请求头，不含鉴权头
error	只在 outcome 不是 success 时出现，否则为 null 。 hint 为错误说明，没有时为 null
response	客户端实际收到的响应正文，没有保存时为 null 。失败请求为错误正文，最多 16 KiB；成功请求为原样转发的正文（流式请求是 SSE 事件流 event: …\ndata: … ），最多 64 KiB，只保留最近 7 天
response.encoding	utf-8 时 body 为文本； base64 时 body 为原始字节的 Base64（正文不是合法 UTF-8，例如仍带压缩）
response.content_encoding	正文仍带的压缩编码；空串表示未压缩（gzip 已解开）
response.size	实际发送给客户端的字节数，正文被截断时大于 body 的长度

跳转至主要内容

字段	说明
`response.truncated / complete`	`truncated` 为 `true` 表示超出上限、只保存了前面部分； `complete` 为 `false` 表示正文未完整保存（被截断，或传输中断）
`cost`	按记录时的价格估算的费用（美元，十进制字符串）；没有用量或模型未定价时为 `null`
`timing.ttft_ms`	流式请求到首个有效内容的耗时；非流式、失败的请求为 `null`

请求日志保留天数与控制台一致，超过保留期的请求查不到。

查询封禁原因 未勾选

`POST` `/open/v1/bans/query` scope `bans:read`

按 session ID 查自己下游会话当前的封禁状态和原因，与控制台「封禁原因查询」一致。

参数

查询封禁原因的参数 复制

```
{"session_id": "4f1c…"}
```

`session_id` 必填，最长 256 字节。该 session 必须经你的 Key 转发过请求，否则返回 `404 SESSION_NOT_FOUND`，不区分 session 不存在还是属于他人。

返回的 data

查询封禁原因的返回示例 复制

```
{
  "session_id": "4f1c…",
  "details_restricted": false,
  "session_bans": [{
    "banned_at": "2026-10-14T01:00:00Z",
    "source": "rule",
    "reason": "该会话此前触发过 cyber 安全限制",
    "categories": ["cyber"]
  }],
  "device_bans": [{
    "banned_at": "2026-10-14T01:00:00Z",
```

跳转至主要内容

```
    "reasons": ["人工确认风险"],
    "categories": ["distill"],
    "labels": ["批量调用"]
  }]
}
```

字段	说明
session_bans	针对该 session 的封禁
session_bans[].source	rule 规则命中； manual 手动封禁； upstream_refusal 上游拒绝触发
device_bans	该 session 所用设备被封禁，因而影响该会话
categories	风险分类代码，见下表； labels 为补充标签
banned_at	封禁时间（UTC）；未知时为 null
details_restricted	为 true 时，部分依据涉及其他用户也在用的 session 或设备，不返回：此时不查询 session 级封禁，受影响设备的 reasons 、 categories 、 labels 、 banned_at 均为 null 。如需了解详情请联系管理员

session_bans 与 device_bans 都为空、且 details_restricted 为 false 时，表示该会话当前没有被封禁。

风险分类代码

cyber 网络安全	reasoning_extraction 推理提取	politics 政治风险
illegal 违法内容	crack 破解	reverse 逆向工程
porn 色情	distill 模型蒸馏	datasynth 数据合成
violence 暴力	hate 仇恨歧视	privacy 隐私
advertising 广告		

以后可能增加新的代码，不认识的代码请原样展示。

推送 规划中

，格式先行固定，届时不需要更换 SK 或解密函数：

跳转至主要内容

推送示例 复制

```
POST <你登记的回调地址>
X-Gw-Access-Key: gwak_…
X-Gw-Timestamp:  1792647000
X-Gw-Event-Id:   evt_5b2e9d0c7a314f68

{"payload": "<base64(iv || ciphertext || tag)>"}
```

用 `resp_key` 解密，AAD 为 `GW1-HOOK\n{AK}\n{timestamp}\n{event_id}` 。收到后检查时间戳在 5 分钟以内，按 `event_id` 去重（可能重复送达），尽快返回 2xx。

常见问题

一直返回 `AUTH_FAILED`

先跑示例的 `selftest` ，或用测试向量逐项核对 `req_key` 、AAD 和 payload。常见原因：SK 按标准 base64 而不是 base64url 解码；salt 用了 SK 而不是 AK；AAD 末尾多了换行；签名用的路径与实际请求路径不一致；SK 已被重置。

`TIMESTAMP_EXPIRED`

校准服务器时间（NTP），或参考响应头 `X-Gw-Server-Time` 。

`NONCE_REPLAYED`

每个请求都要生成新的 nonce，重试时也要换。

响应解密失败

响应 AAD 用的是本次请求的 nonce，不是响应里的任何字段。